

EXPERIMENT NO. 2

REAL-TIME STREAM ANALYTICS AND ANOMALY DETECTION USING NODE-RED AND MQTT

ABISHEK DEVARAJ(192512126)

AIM

To design and implement a real-time stream analytics system using the MQTT protocol and perform anomaly detection on sensor data. Google Colab is used for MQTT communication, stream visualization, data analysis, and anomaly detection.

OBJECTIVES

1. To generate real-time IoT sensor data.
2. To publish sensor values using the MQTT protocol.
3. To subscribe to MQTT messages.
4. To store and analyze incoming sensor data.
5. To detect abnormal sensor values.
6. To calculate statistical parameters such as mean and moving average.
7. To display real-time sensor graphs.
8. To store the collected data for future analysis.

SOFTWARE / APPARATUS REQUIRED

- Google Colab
- Python 3
- Public MQTT Broker – HiveMQ
- Paho MQTT Library
- NumPy
- Pandas
- Matplotlib

THEORY

MQTT (Message Queuing Telemetry Transport) is a lightweight communication protocol widely used in Internet of Things (IoT) applications. It follows a publish-subscribe communication model.

The basic MQTT architecture is:

Publisher → MQTT Broker → Subscriber

The publisher generates sensor data and sends it to the MQTT broker through a specific topic. The subscriber subscribes to the same topic and receives the sensor data.

In a conventional IoT system, Node-RED can be used for subscribing to MQTT data, processing the stream, and displaying the results through a dashboard. In this experiment, Google Colab and Python are used to simulate the stream analytics functionality.

The experiment uses two sensor parameters:

- Temperature
- Humidity

Statistical analytics such as mean and moving average are calculated from the received data.

ANOMALY DETECTION

For this experiment, the normal temperature range is considered as:

$$18^{\circ}\text{C} \leq \text{Temperature} \leq 34^{\circ}\text{C}$$

The anomaly condition is:

$$\text{Temperature} > 34^{\circ}\text{C} \text{ OR } \text{Temperature} < 18^{\circ}\text{C}$$

If the temperature satisfies either condition, the corresponding reading is marked as an **Anomaly**. Otherwise, it is marked as **Normal**.

MQTT COMMUNICATION

Publisher:

Generates temperature and humidity readings and publishes them to the MQTT broker.

MQTT Broker:

Receives the published messages and forwards them to subscribed clients.

Subscriber:

Receives the sensor messages and performs data storage and analytics.

Topic:

IoT_AI_LAB/RealtimeAnalytics_02

ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Define the MQTT broker, port, and topic.
4. Connect the publisher to the MQTT broker.
5. Generate random temperature and humidity values.
6. Publish the generated sensor values as JSON messages.
7. Connect the subscriber to the same MQTT broker.
8. Subscribe to the specified MQTT topic.
9. Receive the incoming sensor messages.
10. Decode the received JSON data.
11. Store temperature, humidity, and time values.
12. Calculate average temperature and humidity.
13. Calculate the moving average of temperature.
14. Check each temperature value against the normal range.
15. Mark abnormal values as anomalies.
16. Display the latest sensor readings.
17. Plot the temperature and humidity values in real time.
18. Continue the process until the program is stopped.

PROGRAM

```
# =====
# EXPERIMENT 2
# REAL-TIME STREAM ANALYTICS AND ANOMALY DETECTION USING MQTT
# Google Colab Version
# =====

# ----- Install MQTT Library -----
!pip -q install paho-mqtt

# ----- Import Libraries -----
import paho.mqtt.client as mqtt
import json
import random
import threading
import time
import pandas as pd
import matplotlib.pyplot as plt

from IPython.display import clear_output

# =====
# MQTT CONFIGURATION
# =====

BROKER = "broker.hivemq.com"
PORT = 1883

# Different topic for this experiment/sample
TOPIC = "IoT_AI_LAB/RealtimeAnalytics_02"

# =====
# DATA STORAGE
# =====

temperature_data = []
humidity_data = []
timestamps = []

# =====
# PUBLISHER
# =====

publisher = mqtt.Client()

def start_publisher():
    try:
        publisher.connect(BROKER, PORT)
```

```

print("Publisher connected to MQTT broker")

count = 0

while True:

    # -----
    # Generate temperature values
    # Normal temperature range: 21 to 33 °C
    # Every 7th sample is intentionally abnormal
    # -----

    if count % 7 == 0:
        temperature = round(
            random.uniform(35, 39),
            2
        )
    else:
        temperature = round(
            random.uniform(21, 33),
            2
        )

    # -----
    # Generate humidity values
    # -----

    humidity = round(
        random.uniform(45, 85),
        2
    )

    # -----
    # Create JSON payload
    # -----

    payload = {
        "temperature": temperature,
        "humidity": humidity
    }

    # -----
    # Publish message
    # -----

    publisher.publish(
        TOPIC,
        json.dumps(payload)
    )

    count += 1

    time.sleep(1)

```

```

        except Exception as e:
            print("Publisher Error:", e)

# =====
# SUBSCRIBER
# =====

subscriber = mqtt.Client()

def on_connect(client, userdata, flags, rc):

    if rc == 0:

        print("Subscriber connected to MQTT broker")

        client.subscribe(TOPIC)

        print("Subscribed to:", TOPIC)

    else:

        print(
            "Subscriber connection failed. Error code:",
            rc
        )

def on_message(client, userdata, msg):

    try:

        # Decode MQTT message
        data = json.loads(
            msg.payload.decode()
        )

        # Extract sensor values
        temperature = data["temperature"]
        humidity = data["humidity"]

        # Store data
        temperature_data.append(
            temperature
        )

        humidity_data.append(
            humidity
        )

        timestamps.append(
            len(timestamps) + 1
        )

```

```

except Exception as e:

    print(
        "Message processing error:",
        e
    )

# Assign callback functions
subscriber.on_connect = on_connect
subscriber.on_message = on_message

# =====
# CONNECT SUBSCRIBER
# =====

try:

    subscriber.connect(
        BROKER,
        PORT
    )

except Exception as e:

    print(
        "Subscriber connection error:",
        e
    )

# =====
# START PUBLISHER THREAD
# =====

publisher_thread = threading.Thread(
    target=start_publisher,
    daemon=True
)

publisher_thread.start()

# =====
# START SUBSCRIBER THREAD
# =====

subscriber_thread = threading.Thread(
    target=subscriber.loop_forever,
    daemon=True
)

subscriber_thread.start()

```

```

# =====
# WAIT FOR MQTT DATA
# =====

print("Waiting for sensor data...")

while len(temperature_data) < 5:

    time.sleep(1)

# =====
# REAL-TIME STREAM ANALYTICS
# =====

try:

    while True:

        # Make a copy of current data
        temp_copy = temperature_data.copy()
        hum_copy = humidity_data.copy()
        time_copy = timestamps.copy()

        # -----
        # Create DataFrame
        # -----

        df = pd.DataFrame({

            "Time": time_copy,

            "Temperature": temp_copy,

            "Humidity": hum_copy

        })

        # -----
        # Calculate Mean
        # -----

        average_temperature = (
            df["Temperature"].mean()
        )

        average_humidity = (
            df["Humidity"].mean()
        )

        # -----
        # Calculate Moving Average

```

```

# Window size = 5
# -----

df["Moving_Average"] = (
    df["Temperature"]
    .rolling(window=5)
    .mean()
)

# -----
# Anomaly Detection
#
# Normal temperature:
# 18 °C to 34 °C
#
# Anomaly:
# Temperature > 34 °C
# OR
# Temperature < 18 °C
# -----

df["Status"] = df["Temperature"].apply(
    lambda x:
        "Anomaly"
        if x > 34 or x < 18
        else "Normal"
)

# -----
# Clear Previous Output
# -----

clear_output(
    wait=True
)

# =====
# DISPLAY RESULTS
# =====

print("=" * 70)

print(
    "REAL-TIME STREAM ANALYTICS USING MQTT"
)

print("=" * 70)

print()

```



```

print(
    "MQTT Broker :",
    BROKER
)

print(
    "MQTT Topic  :",
    TOPIC
)

print()

print(
    "Latest Sensor Readings:"
)

print()

print(
    df.tail(10).to_string(
        index=False
    )
)

print()

print("-" * 70)

print(
    "Average Temperature :",
    round(
        average_temperature,
        2
    ),
    "°C"
)

print(
    "Average Humidity    :",
    round(
        average_humidity,
        2
    ),
    "%"
)

print(
    "Maximum Temperature :",
    round(
        df["Temperature"].max(),
        2
    ),
    "°C"
)

```

```

print(
    "Minimum Temperature :",
    round(
        df["Temperature"].min(),
        2
    ),
    "°C"
)

print(
    "Total Messages      :",
    len(df)
)

anomaly_count = (
    df["Status"] == "Anomaly"
).sum()

print(
    "Anomalies Detected  :",
    anomaly_count
)

print("-" * 70)

# =====
# REAL-TIME GRAPH
# =====

plt.figure(
    figsize=(12, 6)
)

# Temperature graph
plt.plot(

    df["Time"],

    df["Temperature"],

    marker="o",

    label="Temperature (°C)"

)

# Humidity graph
plt.plot(

    df["Time"],

```

```

        df["Humidity"],

        marker="s",

        label="Humidity (%)"
    )

# -----
# Select Anomalies
# -----

anomaly_data = df[
    df["Status"] == "Anomaly"
]

# -----
# Mark Anomalies
# -----

if len(anomaly_data) > 0:

    plt.scatter(

        anomaly_data["Time"],

        anomaly_data["Temperature"],

        color="red",

        s=100,

        label="Temperature Anomaly"

    )

# -----
# Graph Formatting
# -----

plt.title(
    "Real-Time Temperature and Humidity Analytics"
)

plt.xlabel(
    "Time / Sample Number"
)

plt.ylabel(
    "Sensor Values"
)

```

```

plt.grid(
    True
)

plt.legend()

plt.tight_layout()

plt.show()

# -----
# Wait before next update
# -----

time.sleep(2)

except KeyboardInterrupt:

    print(
        "Real-time analytics stopped."
    )

except Exception as e:

    print(
        "Analytics Error:",
        e
    )

```

SAMPLE OUTPUT

The output will continuously update as new MQTT messages are received.

Example:

```

=====
REAL-TIME STREAM ANALYTICS USING MQTT
=====

```

Latest Sensor Readings:

Time	Temperature	Humidity	Moving_Average	Status
21	27.84	63.42	28.16	Normal
22	31.27	71.83	29.07	Normal
23	25.63	56.21	28.52	Normal
24	37.46	79.34	30.14	Anomaly
25	29.18	68.72	30.28	Normal

```

Average Temperature : 28.91 ° C
Average Humidity    : 66.47 %
Maximum Temperature : 37.46 ° C
Minimum Temperature : 21.35 ° C
Total Messages      : 25
Anomalies Detected  : 3

```

GRAPH OUTPUT

The Google Colab output displays a real-time graph containing

- Temperature readings
- Humidity readings
- Sample/time number on the X-axis
- Sensor values on the Y-axis
- Anomalous temperature readings highlighted separately

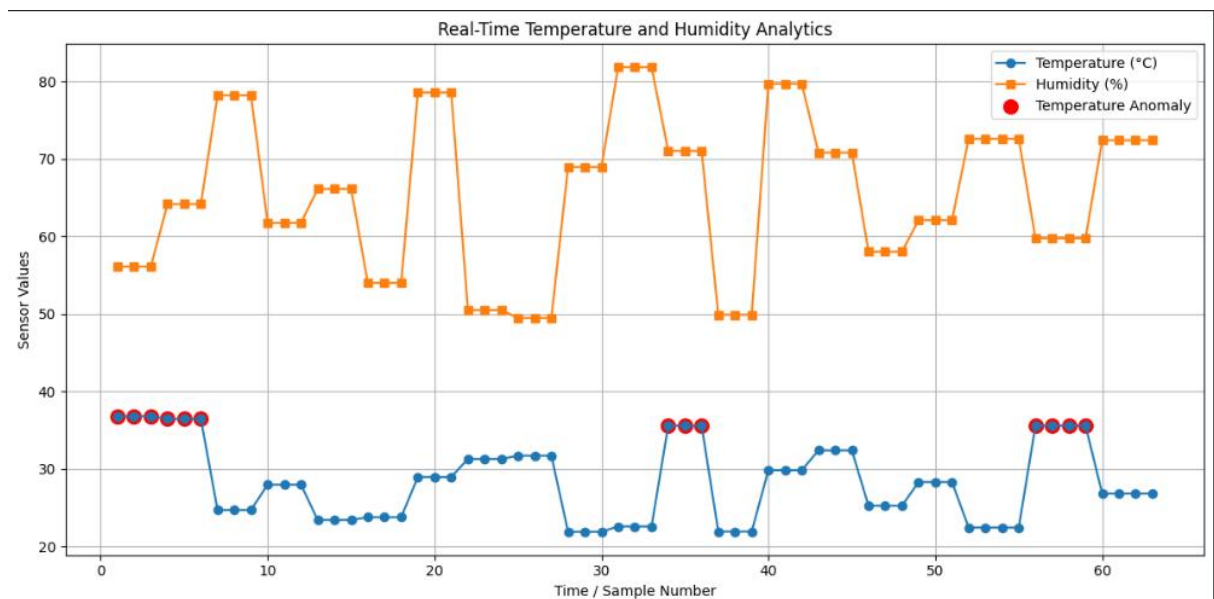
The graph changes continuously as new MQTT sensor messages arrive.

OBSERVATION

S.No.	Temperature (° C)	Humidity (%)	Status
1	27.84	63.42	Normal
2	31.27	71.83	Normal
3	25.63	56.21	Normal
4	37.46	79.34	Anomaly
5	29.18	68.72	Normal
6	23.91	52.64	Normal
7	35.72	74.15	Anomaly
8	28.47	61.38	Normal
9	32.16	80.27	Normal
10	36.81	69.45	Anomaly

The actual values will vary each time the program is executed because the sensor readings are generated randomly.

OUTPUT :



RESULT

The real-time IoT stream analytics system was successfully implemented using the MQTT protocol and Google Colab. Sensor data consisting of temperature and humidity values was generated and transmitted through the MQTT broker. The subscriber successfully received and stored the incoming data.

Statistical analysis including average, minimum, maximum, and moving average was performed. Temperature anomalies were successfully detected when the readings exceeded the defined normal range. The sensor data and detected anomalies were also visualized using real-time graphs in Google Colab.

Thus, the experiment demonstrates the implementation of **MQTT-based real-time stream analytics and anomaly detection for IoT sensor data** without requiring a local Node-RED installation.